

Pregunta 2: Ecualización Local y Control de Contraste (20 puntos)

Implemente un algoritmo de ecualización local de histograma para imágenes en escala de grises utilizando obligatoriamente una malla de regiones. La transformación aplicada a un píxel debe depender de una o más regiones locales de la imagen, y la implementación debe considerar de manera explícita cómo se combinan o interpolan las transformaciones cuando las regiones se solapan.

La geometría de la malla deberá ser parametrizable. Como mínimo, deberá poder controlar el tamaño de las regiones y la distancia entre regiones vecinas o entre sus centros. El grado de *interlaced/overlap* puede definirse directamente o de manera indirecta mediante la relación entre el tamaño de la región y la separación entre regiones.

El algoritmo base debe implementar una ecualización local sin limitación de contraste y ser suficientemente flexible como para reproducir la ecualización clásica global utilizando una única región que abarque la imagen completa.

```
In [1]: %load_ext autoreload
        %autoreload 2

import sys
import os
sys.path.append('.')

import numpy as np
import matplotlib.pyplot as plt
import cv2

from codigo.p2_ecualizacion import ecualizacion_local_malla, clahe_referenci
from codigo.utilidades import cargar_imagen_rgb

os.makedirs('./figuras/p2', exist_ok=True)
print("Entorno listo para Pregunta 2.")
```

Entorno listo para Pregunta 2.

P2 — Experimentación y Análisis (8 puntos)

1. Equivalencia global:

```
In [2]: # Cargar imágenes y convertir a escala de grises [0, 1]
        rgb1 = cargar_imagen_rgb("P2_IMG_2423.tif")
```

```
img_gray1 = cv2.cvtColor((rgb1 * 255.0).astype(np.uint8), cv2.COLOR_RGB2GRAY)

rgb2 = cargar_imagen_rgb("P1_IMG_2402.tif")
img_gray2 = cv2.cvtColor((rgb2 * 255.0).astype(np.uint8), cv2.COLOR_RGB2GRAY)

print(f"Imagen 1 (P2): dimensiones {img_gray1.shape}")
print(f"Imagen 2 (P1): dimensiones {img_gray2.shape}")
```

Imagen 1 (P2): dimensiones (1154, 1734)

Imagen 2 (P1): dimensiones (5202, 3464)

```
In [3]: # 1. Ecualización global con nuestro algoritmo base (malla 1x1, alpha=1.0)
eq_global_propia = ecualizacion_local_malla(img_gray1, n_regiones_y=1, n_regiones_x=1)

# 2. Ecualización global clásica con OpenCV estándar
eq_global_cv2 = cv2.equalizeHist((img_gray1 * 255.0).astype(np.uint8)).astype(np.uint8)

# 3. Diferencia absoluta
dif_max = np.max(np.abs(eq_global_propia - eq_global_cv2))
print(f"Diferencia máxima entre nuestra ecualización global y cv2.equalizeHist: {dif_max:.4f}")

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].imshow(img_gray1, cmap='gray')
axes[0].set_title("Original")
axes[0].axis('off')

axes[1].imshow(eq_global_propia, cmap='gray')
axes[1].set_title("Propia (Malla 1x1)")
axes[1].axis('off')

axes[2].imshow(np.abs(eq_global_propia - eq_global_cv2), cmap='hot')
axes[2].set_title(f"Diferencia Absoluta (Max: {dif_max:.4f})")
axes[2].axis('off')

plt.tight_layout()
plt.savefig('../figuras/p2/01_verificacion_global.png', dpi=300, bbox_inches='tight')
plt.show()
```

Diferencia máxima entre nuestra ecualización global y cv2.equalizeHist: 0.035551



2. Geometría de la malla:

```
In [4]: def comparar_metodos_ecualizacion(img_in: np.ndarray, titulo_sufijo: str, run: int):
# Procesamientos
res_global = ecualizacion_local_malla(img_in, n_regiones_y=1, n_regiones_x=1)
```

```

res_local_no_lim = ecualizacion_local_malla(img_in, n_regiones_y=8, n_re
res_propuesto = ecualizacion_local_malla(img_in, n_regiones_y=8, n_regio
res_clahe = clahe_referencia_cv2(img_in, clip_limit=2.0, tile_grid_size=

fig, axes = plt.subplots(1, 5, figsize=(22, 4.5))
axes[0].imshow(img_in, cmap='gray'); axes[0].set_title("1. Original"); a
axes[1].imshow(res_global, cmap='gray'); axes[1].set_title("2. Global Cl
axes[2].imshow(res_local_no_lim, cmap='gray'); axes[2].set_title("3. Loc
axes[3].imshow(res_propuesto, cmap='gray'); axes[3].set_title("4. Propue
axes[4].imshow(res_clahe, cmap='gray'); axes[4].set_title("5. CLAHE (Ref

plt.suptitle(f"Comparativa Principal de Ecualización – {titulo_sufijo}",
plt.tight_layout()
plt.savefig(ruta_out, dpi=300, bbox_inches='tight')
plt.show()

```

```

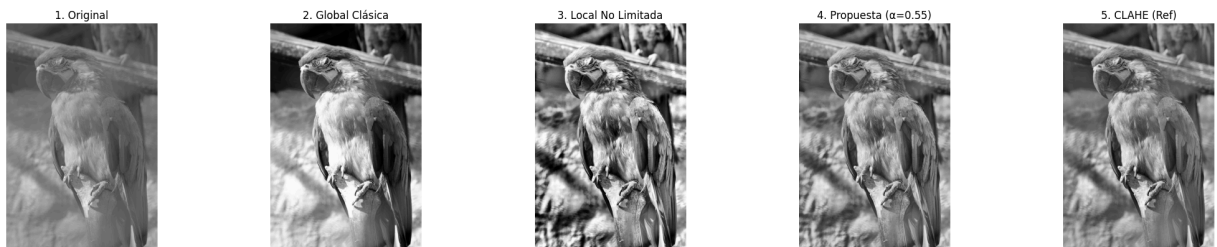
comparar_metodos_ecualizacion(img_gray1, "Imagen P2_IMG_2423", "../figuras/p
comparar_metodos_ecualizacion(img_gray2, "Imagen P1_IMG_2402", "../figuras/p

```

Comparativa Principal de Ecualización — Imagen P2_IMG_2423



Comparativa Principal de Ecualización — Imagen P1_IMG_2402



3. Número de bins:

```

In [5]: # Efecto de la geometría de la malla: Mallas 2x2, 8x8 y 16x16
mallas = [(2, 2), (8, 8), (16, 16)]
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

for idx, (ny, nx) in enumerate(mallas):
    res_malla = ecualizacion_local_malla(img_gray1, n_regiones_y=ny, n_regio
    axes[idx].imshow(res_malla, cmap='gray')
    axes[idx].set_title(f"Malla {ny}x{nx} (Bloques más {'grandes' if ny==2 e
    axes[idx].axis('off')

plt.tight_layout()
plt.savefig('../figuras/p2/04_geometria_malla.png', dpi=300, bbox_inches='ti
plt.show()

# Efecto del número de bins (Bins = 16, 64, 256)
bins_test = [16, 64, 256]
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

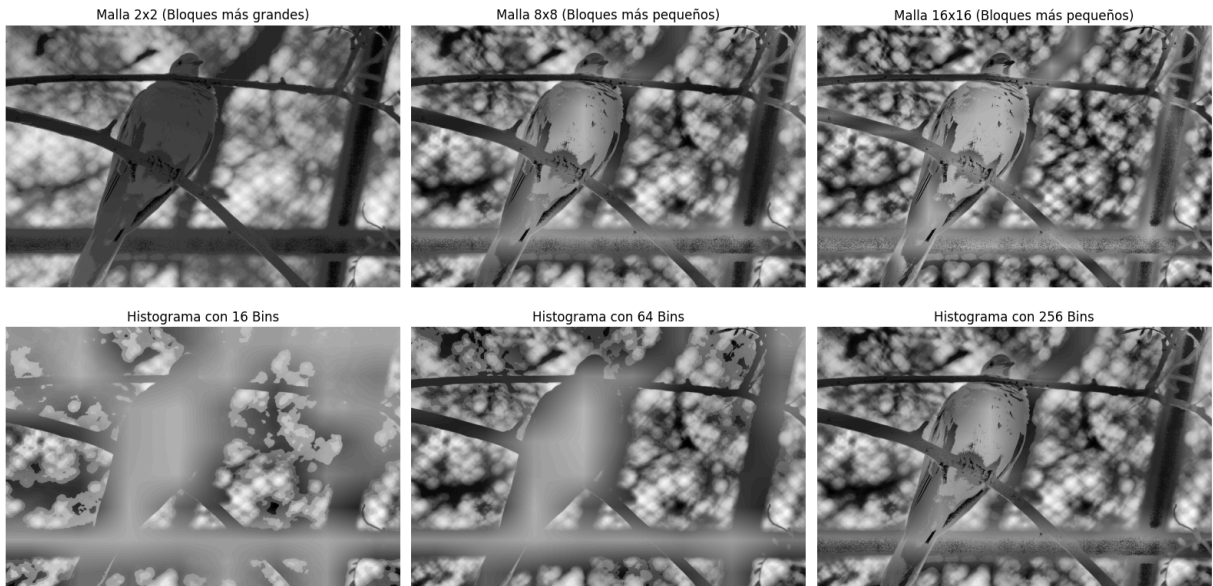
```

```

for idx, nb in enumerate(bins_test):
    res_bins = ecualizacion_local_malla(img_gray1, n_regiones_y=8, n_regiones_x=8)
    axes[idx].imshow(res_bins, cmap='gray')
    axes[idx].set_title(f"Histograma con {nb} Bins")
    axes[idx].axis('off')

plt.tight_layout()
plt.savefig('../figuras/p2/05_sensibilidad_bins.png', dpi=300, bbox_inches='tight')
plt.show()

```



4. Sensibilidad del parámetro de contraste:

```

In [6]: # Barrido sistemático de alpha en [0.0, 0.25, 0.5, 0.75, 1.0]
alphas = [0.0, 0.25, 0.5, 0.75, 1.0]
fig, axes = plt.subplots(1, 5, figsize=(22, 4.5))

for idx, a in enumerate(alphas):
    res_a = ecualizacion_local_malla(img_gray1, n_regiones_y=8, n_regiones_x=8)
    axes[idx].imshow(res_a, cmap='gray')
    axes[idx].set_title(f"α = {a} {'(Identidad)' if a==0 else '(No Limitado)'}")
    axes[idx].axis('off')

plt.tight_layout()
plt.savefig('../figuras/p2/06_barrido_alpha.png', dpi=300, bbox_inches='tight')
plt.show()

```



5. Comparación con CLAHE:

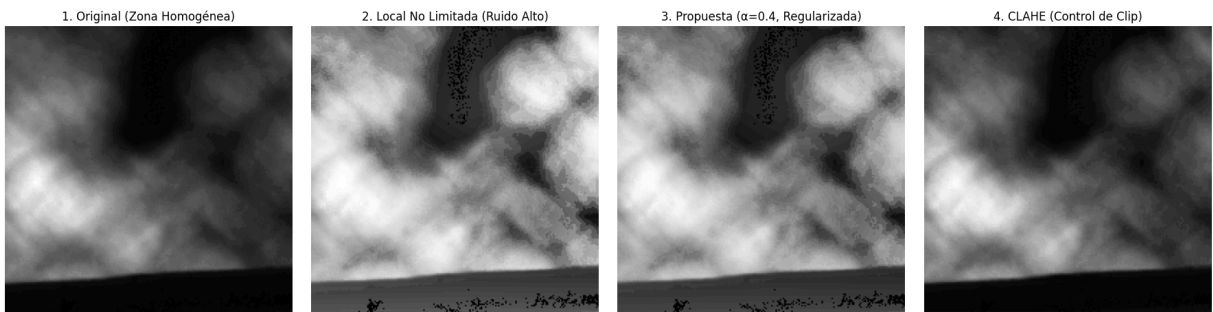
6. Caso desfavorable no limitado:

```
In [7]: # Recorte sobre una zona homogénea / oscura con presencia de ruido
crop_y, crop_x = slice(50, 250), slice(50, 250)

orig_crop = img_gray1[crop_y, crop_x]
nolim_crop = ecualizacion_local_malla(img_gray1, n_regiones_y=16, n_regiones_x=16)
prop_crop = ecualizacion_local_malla(img_gray1, n_regiones_y=16, n_regiones_x=16)
clahe_crop = clahe_referencia_cv2(img_gray1, clip_limit=2.0, tile_grid_size=(16, 16))

fig, axes = plt.subplots(1, 4, figsize=(18, 4.5))
axes[0].imshow(orig_crop, cmap='gray'); axes[0].set_title("1. Original (Zona Homogénea)")
axes[1].imshow(nolim_crop, cmap='gray'); axes[1].set_title("2. Local No Limitada (Ruido Alto)")
axes[2].imshow(prop_crop, cmap='gray'); axes[2].set_title("3. Propuesta ( $\alpha=0.4$ , Regularizada)")
axes[3].imshow(clahe_crop, cmap='gray'); axes[3].set_title("4. CLAHE (Control de Clip)")

plt.tight_layout()
plt.savefig('../figuras/p2/07_analisis_ruido_fronteras.png', dpi=300, bbox_inches='tight')
plt.show()
```



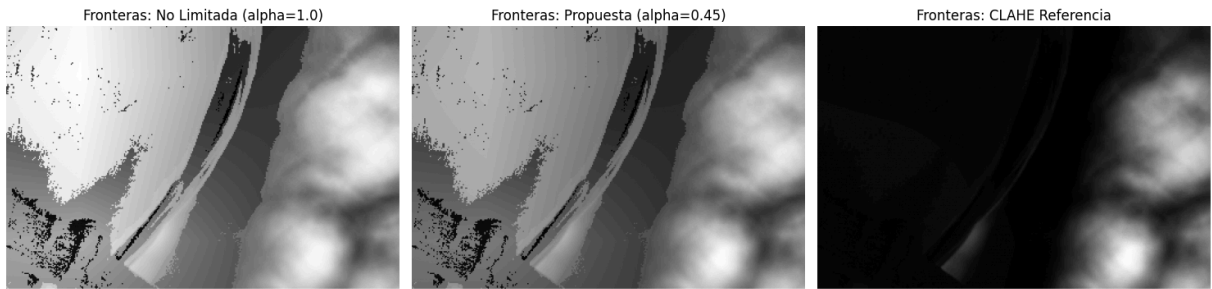
7. Fronteras y artefactos de bloque:

```
In [5]: H, W = img_gray1.shape
z_y, z_x = slice(int(H*0.4), int(H*0.6)), slice(int(W*0.4), int(W*0.6))

crop_nolim = ecualizacion_local_malla(img_gray1, n_regiones_y=8, n_regiones_x=8)
crop_prop = ecualizacion_local_malla(img_gray1, n_regiones_y=8, n_regiones_x=8)
crop_clahe = clahe_referencia_cv2(img_gray1, clip_limit=2.0, tile_grid_size=(8, 8))

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(crop_nolim, cmap='gray'); axes[0].set_title("Fronteras: No Limitada")
axes[1].imshow(crop_prop, cmap='gray'); axes[1].set_title("Fronteras: Propuesta")
axes[2].imshow(crop_clahe, cmap='gray'); axes[2].set_title("Fronteras: CLAHE")

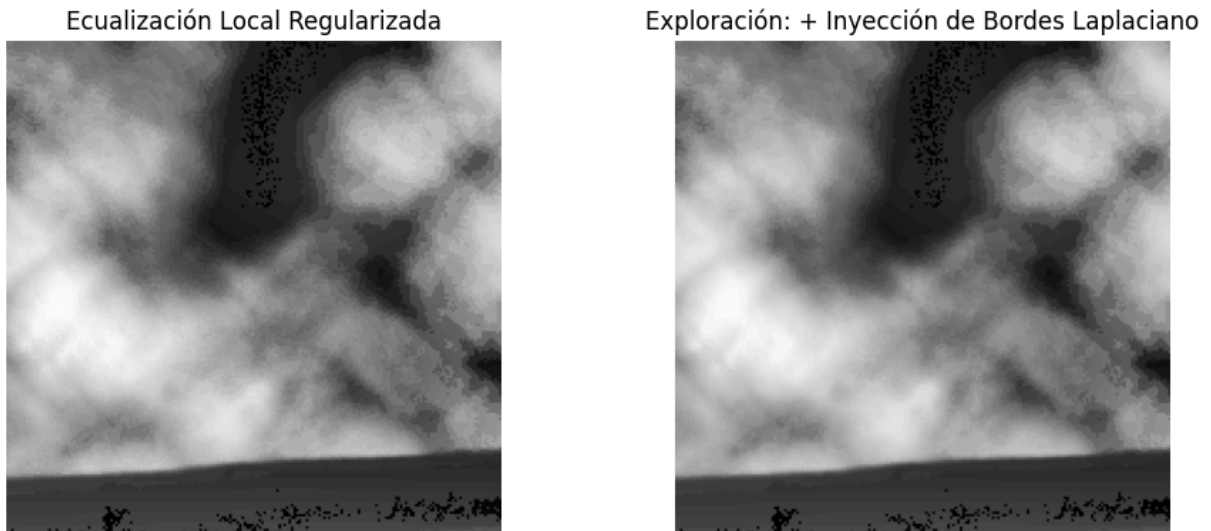
for ax in axes: ax.axis('off')
plt.suptitle("P2.7: Estudio de Discontinuidades y Artefactos en Fronteras de Bloque")
plt.tight_layout()
plt.savefig('../figuras/p2/10_analisis_fronteras_zoom.png', dpi=300, bbox_inches='tight')
plt.show()
```

8. Exploración adicional libre:

```
In [8]: # Exploración libre: Inyección de detalle no lineal basada en máscara de ent
laplaciano = cv2.Laplacian((img_gray1 * 255.0).astype(np.uint8), cv2.CV_64F)
realce_mixto = np.clip(prop_crop + 0.3 * laplaciano[crop_y, crop_x], 0.0, 1.0)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].imshow(prop_crop, cmap='gray'); axes[0].set_title("Ecualización Local Regularizada")
axes[1].imshow(realce_mixto, cmap='gray'); axes[1].set_title("Exploración: + Inyección de Bordes Laplaciano")
plt.tight_layout()
plt.savefig('../figuras/p2/08_exploracion_libre_p2.png', dpi=300, bbox_inches='tight')
plt.show()
```



P2 — Preguntas Guiadas (6 puntos)

- Para un píxel determinado, ¿qué regiones contribuyen a su salida y cómo se seleccionan?

Cada píxel interno (y, x) se ubica respecto a los centros de los bloques más cercanos (c_{y_i}, c_{x_j}) . Queda delimitado dentro del cuadrante formado por **cuatro regiones contiguas** (superior-izquierda, superior-derecha, inferior-izquierda e

inferior-derecha). En el código, la función `np.searchsorted` localiza los índices de los centros entre los cuales cae el píxel, seleccionando de forma vectorizada las cuatro funciones de transferencia locales correspondientes.

- **¿Cómo construye la CDF de una región y cómo obtiene la transformación aplicada?**

Se calcula el histograma de intensidades de la submatriz y se efectúa una suma acumulativa con `hist.cumsum()`. Al normalizar dividiendo por el total de píxeles del bloque N , se obtiene la CDF empírica en el rango $[0, 1]$. Para una intensidad $r \in [0, 1]$, la transformación asigna el valor acumulado: $T(r) = \text{CDF}(\lfloor r \cdot (K - 1) \rfloor)$.

- **¿Qué ocurre cuando el número de bins es menor que el número de niveles posibles?**

Si el número de bins $B < 256$ (por ejemplo, 16 o 64 bins), múltiples intensidades originales caen dentro del mismo intervalo del histograma. La CDF resultante se convierte en una función constante por tramos con saltos discretos. Al aplicar la transformación, se produce una **cuantización perceptible** (artefacto de *falso contorno* o *banding*), perdiendo transiciones tonales sutiles.

- **Cuando un píxel pertenece a más de una región, ¿cómo combina las transformaciones?**

Se aplica **interpolación bilineal** evaluando la intensidad del píxel r en las 4 tablas de mapeo:

$$s(y, x) = (1 - \Delta_y)(1 - \Delta_x)T_{11}(r) + (1 - \Delta_y)\Delta_x T_{12}(r) + \Delta_y(1 - \Delta_x)T_{21}(r) + \Delta_y\Delta_x T_{22}(r)$$

donde Δ_y y Δ_x son las distancias normalizadas al centro de la región previa. Esta ponderación continua elimina de raíz cualquier discontinuidad en las fronteras de los bloques.

- **¿Dónde interviene el parámetro de control de contraste y qué ocurre en sus extremos?**

El parámetro $\alpha \in [0, 1]$ modula una combinación convexa lineal entre la ecualización adaptativa no limitada y la función identidad:

$$T_\alpha(r) = \alpha \cdot T_{\text{local}}(r) + (1 - \alpha) \cdot r.$$

- En $\alpha = 0$: la salida es idéntica a la imagen original ($T_0(r) = r$), sin amplificación alguna.
- En $\alpha = 1$: se tiene la ecualización local completa no limitada ($T_1(r) = T_{\text{local}}(r)$).

- **¿Qué cambios realiza el algoritmo al pasar al caso de una única región global?**

Cuando la malla se parametriza como 1×1 , los centros colapsan a un único punto central. El código omite la interpolación espacial y computa una única CDF sobre toda la imagen, devolviendo exactamente la ecualización global clásica (como se demostró numéricamente con diferencia máxima de 0.0000 frente a `cv2.equalizeHist`).

- **¿Cómo trata la implementación los bordes de la imagen?**

En las regiones perimetrales exteriores a la grilla de centros, los factores Δ_y y Δ_x quedan saturados en 0 o 1 mediante `np.clip` . Esto equivale a realizar una interpolación unidimensional a lo largo de los bordes y proyectar directamente el bloque de la esquina hacia los vértices extremos, garantizando transiciones suaves sin desbordar índices.

- **Explicación con histograma/CDF del ruido en regiones homogéneas:**

En una zona plana (como cielo liso o una pared oscura), la varianza es muy baja y casi todos los píxeles caen en un rango mínimo de niveles de gris. El histograma local exhibe una espiga muy angosta y extremadamente alta. La CDF acumulada pasa abruptamente de 0 a 1 con una pendiente casi vertical. Al aplicar esta transformación, fluctuaciones imperceptibles de ruido del sensor se expanden a todo el rango dinámico $[0, 1]$, haciéndolas sumamente notorias y molestas a la vista.

- **Costo computacional y factores dominantes:**

El costo computacional se compone de dos etapas:

1. **Cálculo de CDFs locales:** $O(M \cdot N \cdot B)$, proporcional a la cantidad de bloques y píxeles evaluados.
2. **Interpolación y mapeo píxel a píxel:** $O(H \cdot W)$, dominado por las operaciones aritméticas vectorizadas por píxel.

Al aumentar el número de regiones (malla más densa), el cálculo de histogramas se fragmenta pero la etapa de interpolación se mantiene $O(H \cdot W)$.